Министерство науки и высшего образования РФ
АРЗАМАССКИЙ ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ (ФИЛИАЛ)
ФЕДЕРАЛЬНОГО ГОСУДАРСТВЕННОГО БЮДЖЕТНОГО
ОБРАЗОВАТЕЛЬНОГО УЧРЕЖДЕНИЯ ВЫСШЕГО ОБРАЗОВАНИЯ «НИЖЕГОРОДСКИЙ
ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ ИМ. Р.Е.АЛЕКСЕЕВА»
(АПИ НГТУ)

Лабораторная работа №3 по дисциплине «Логика и архитектура вычислительных средств»

Выполнила:
студентка гр. МА26ПМ
Фролова Д.А.

Проверил:
Комаров А.О.

Арзамас, 2026

Анализ отзывов пациентов о клиниках

1 Цель, требования и ограничения

Цель проекта - автоматически выделять в отзыве аспекты «врач», «ожидание», «сервис», «стоимость» и «результат лечения», а затем определять тональность по каждому аспекту. Это позволяет клинике видеть причины негативных оценок, не заменяя итоговое решение специалиста.

Требования и ограничения проекта представлены в таблице 1.

Таблица 1 - Требования проекта

Категория	**Содержание**
Функциональные требования	Принять отзыв, выделить аспекты, определить тональность, сохранить результат и показать агрегаты.
Входные данные	review_id, clinic_id, текст отзыва и рейтинг от 1 до 5; время присваивает сервис.
Выходные данные	Список аспектов, тональность, уверенность классификации, агрегаты по клинике.
Ограничения	Отзывы должны быть обезличены до отправки; вывод модели нуждается в проверке.

2 Архитектура и обмен сообщениями

На рисунке 1 показана архитектура: клиент передаёт обезличенный отзыв через API, классификатор выделяет аспекты и тональность, PostgreSQL хранит запись, а Elasticsearch индексирует её. API публикует reviews.created; worker группы review-analytics повторно индексирует событие и публикует reviews.indexed. Dashboard получает сводку из PostgreSQL.

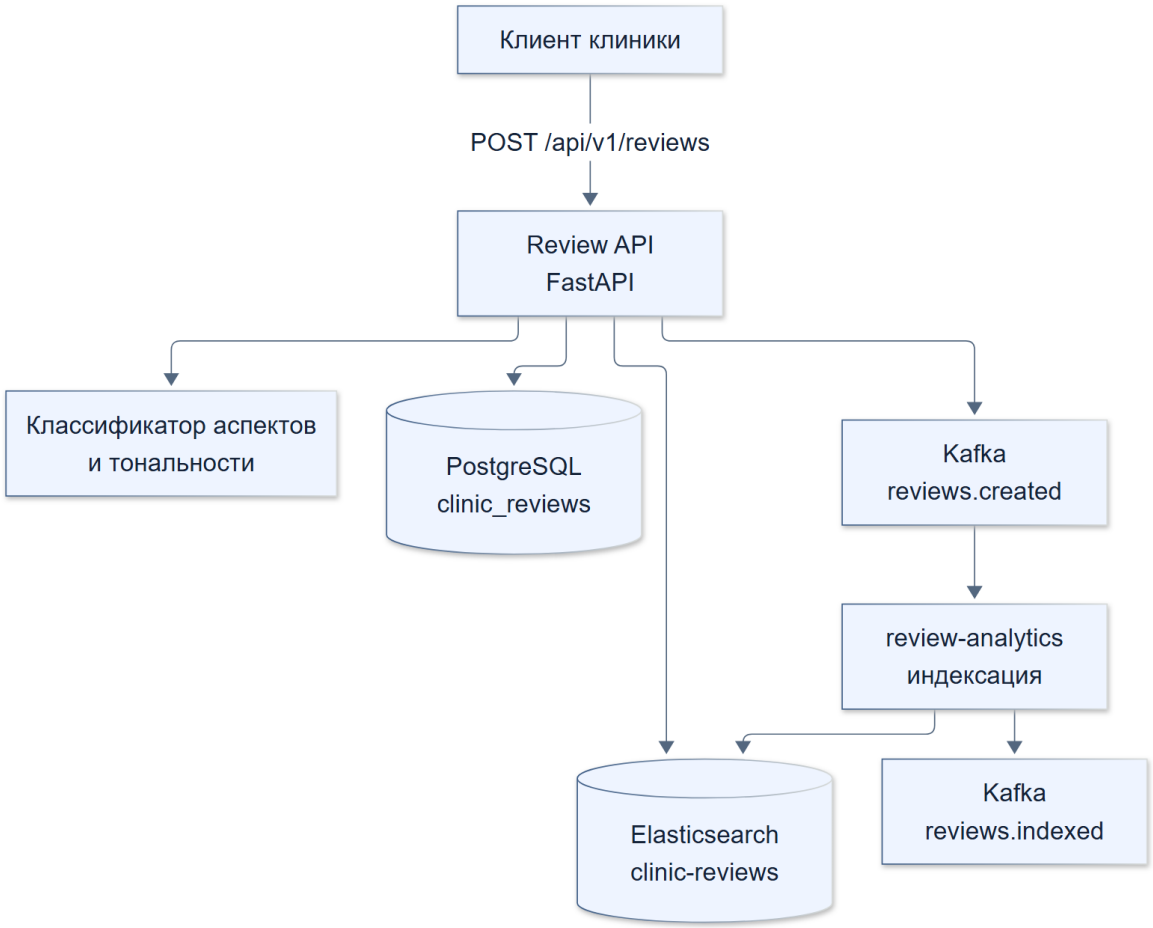


Рисунок 1 - Контейнерная схема анализ отзывов пациентов о клиниках

Поток обработки показан на рисунке 2. Компоненты разделены по ответственности: приём данных, подготовка признаков, интеллектуальная обработка и фиксация результата.

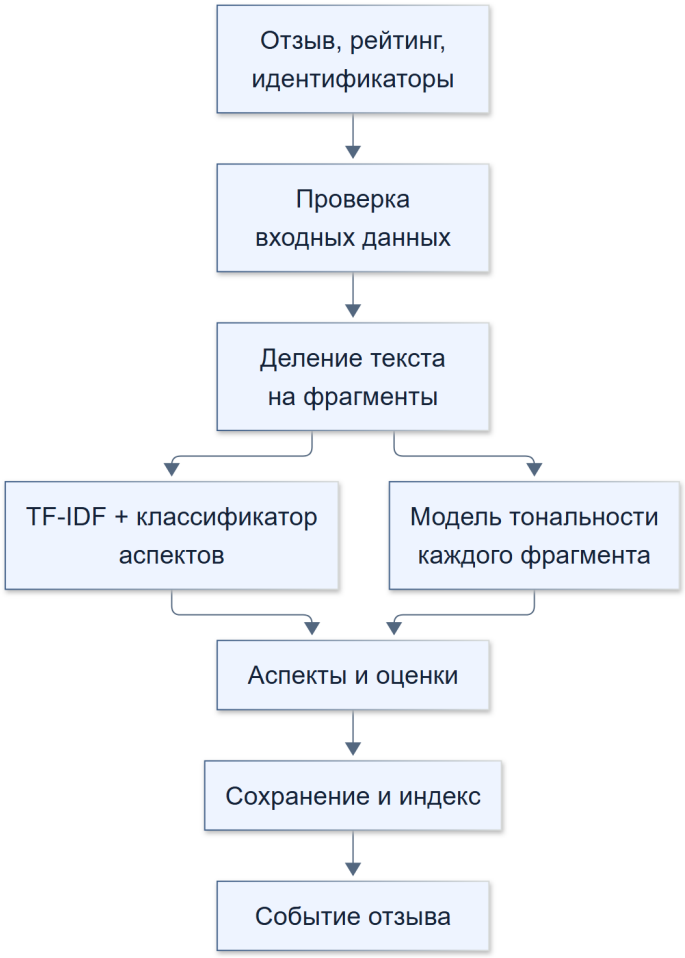


Рисунок 2 - Конвейер обработки данных

Таблица 2 - Каналы взаимодействия

Канал	**Отправитель**	**Получатель**	**Данные**
reviews.created	API	review-analytics	размеченный отзыв
reviews.indexed	review-analytics	внешний потребитель	review_id, index
Kafka	broker	consumer group review-analytics	UTF-8 JSON

3 Данные, API и хранилища

SQLite хранит reviews и labels, индексы clinic_id и sentiment. В PostgreSQL таблица clinic_reviews содержит первичный ключ review_id, clinic_id, rating, text, labels JSONB и created_at; составной индекс clinic_id/created_at поддерживает выборку по клинике. Elasticsearch хранит документы в индексе clinic-reviews по review_id. На рисунке 3 показана исходная логическая схема.

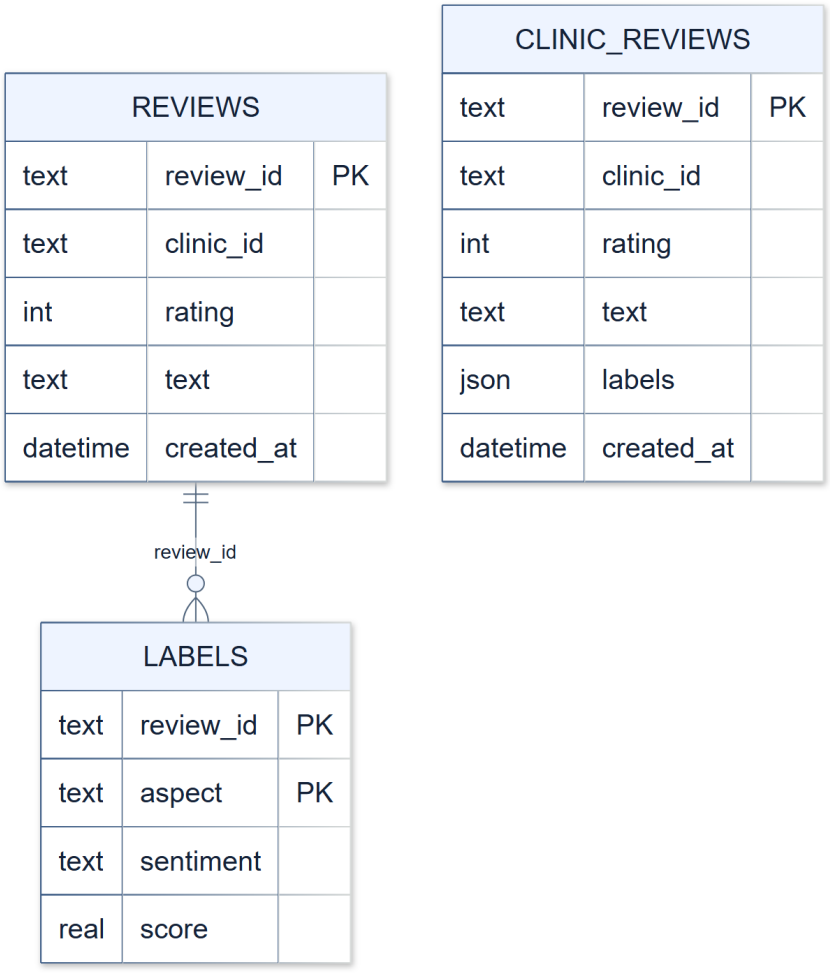


Рисунок 3 - Логическая схема данных

Таблица 3 - Контракт API

Операция	**Назначение**	**Результат**
POST /api/v1/reviews	Добавить отзыв	201, review_id и метки
GET /api/v1/reviews/{id}	Получить разметку	аспекты и тональности
GET /api/v1/clinics/{id}/summary	Агрегаты клиники	Количество пар аспект / тональность
GET /dashboard/{clinic_id}	Аналитическая страница	HTML, группировка результатов
GET /pipeline/{clinic_id}	Этапы обработки	HTML pipeline

```
POST /api/v1/reviews
{"review_id":"rev-103","clinic_id":"cl-
12","rating":2,"text":"Долго ждал приёма, но врач внимательный"}
Ответ 201: review_id, clinic_id, rating, text, labels и
created_at.
```

Листинг 1 - Пример запроса к API

4 Паттерны проектирования

Strategy: BaselineReviewStrategy, CorpusReviewStrategy и TransformerReviewStrategy реализуют classify; настройка REVIEW_CLASSIFIER выбирает режим. Adapter: HuggingFaceReviewClassifier изолирует вызов Transformers. Repository: PostgresReviewRepository сохраняет разметку. Pipeline: ReviewPipeline выполняет нормализацию и классификацию. Observer: LocalReviewObserver записывает отзыв и метки в SQLite.

Код: add_review → review_strategy().classify → PostgresReviewRepository.save → KafkaReviewPublisher.publish (app.py).

Листинг 2 - Компоненты обработки отзыва

5 Реализация, тестирование и результаты

Для оценки используются macro-F1 по аспектам и тональности. Сохранённые результаты отложенной части медицинского корпуса: macro-F1 аспектов 0,5350 против 0,3397 у словарного baseline; тональности 0,7333 против 0,3325. Эта оценка относится к предложениям и не подтверждает точность тональности каждого отдельного аспекта. Результаты проверки сценариев приведены в таблице 4.

Таблица 4 - Результаты контрольных сценариев

Сценарий	**Ожидаемый результат**	**Статус**
Негатив об ожидании	Выделен аспект ожидание	Проверен
Положительный отзыв	Тональность positive	Проверен
Пустой текст	API возвращает 422	Проверен

Рисунок 4 показывает документацию FastAPI при локальном запуске. Результаты модели тональности и классификатора аспектов приведены в приложении Б.

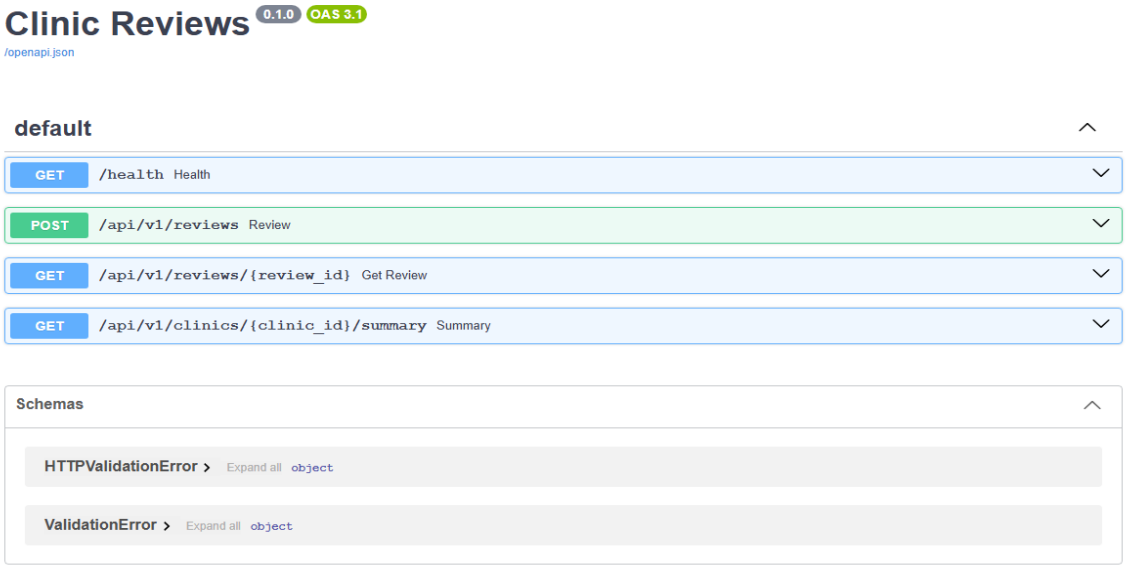


Рисунок 4 - Документация запущенного API

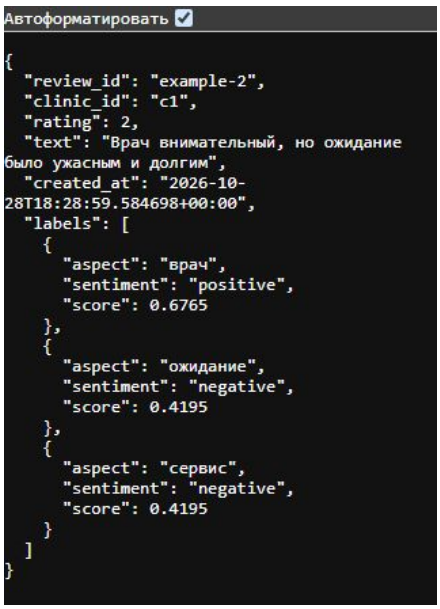


Рисунок 5 - Ответ API при использовании модели корпуса

Проект создаёт аналитический контур от потока текста до сводки по клинике. Результаты классификации требуют проверки специалистом и не предназначены для автоматического принятия медицинских решений.

6 Контекст, компоненты и границы ответственности

Клиент передаёт только заранее обезличенные отзывы. Система принимает поток записей, обрабатывает текст и предоставляет аналитическую сводку. Схемы контекста и компонентов приведены на рисунках 6 и 7.

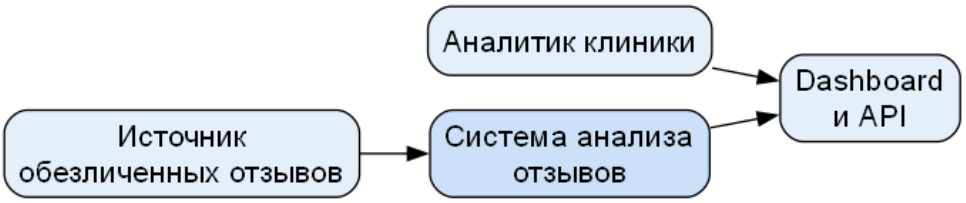


Рисунок 6 - Контекст системы анализа отзывов

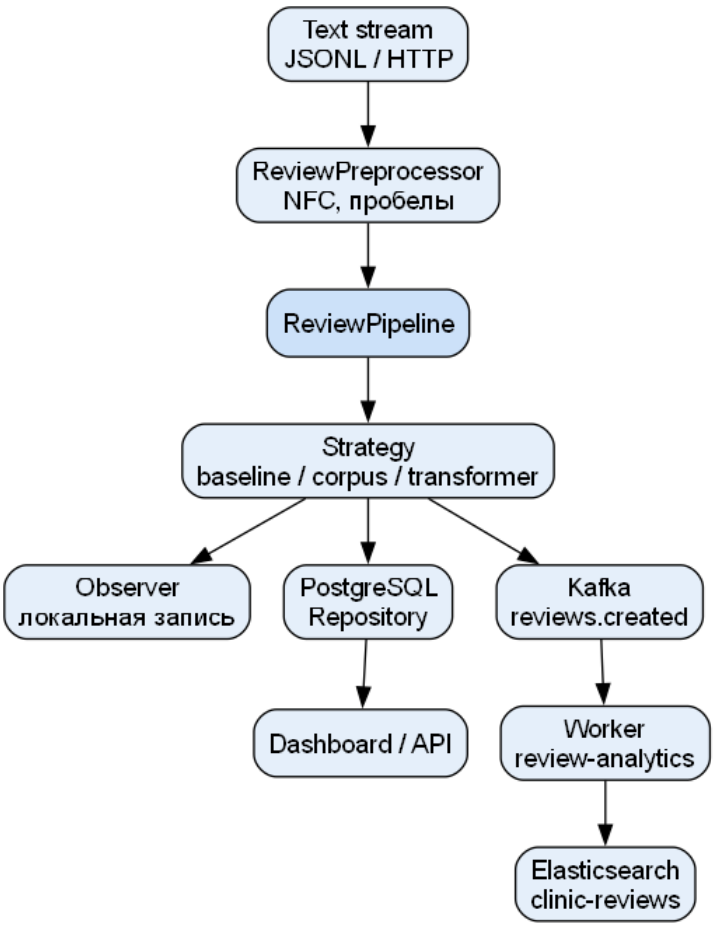


Рисунок 7 - Компоненты обработки и хранения отзывов

Таблица 5 - Ответственность компонентов и сервисов

Компонент	**Назначение**
Text stream / клиент	Передаёт упорядоченные обезличенные отзывы JSONL через review_stream.py или HTTP.
FastAPI и ReviewPipeline	Проверяют вход, нормализуют текст, выбирают классификатор, возвращают результат.
PostgreSQL	Сохраняет отзыв и JSON-метки; обеспечивает чтение и сводку по клинике.
Kafka и worker	reviews.created принимает событие; группа review-analytics индексирует его и публикует reviews.indexed.
Elasticsearch	Индекс clinic-reviews хранит поисковый документ по review_id.
Dashboard	Отображает группировку аспектов и тональности по клинике.

7 Предобработка, контракты и паттерны

ReviewPreprocessor приводит текст к Unicode NFC и нормализует пробельные символы, сохраняя пунктуацию и отрицания. CorpusReviewStrategy делит отзыв на фрагменты по знакам и союзам «но», «однако»; модели TF-IDF и логистической регрессии оценивают аспекты и тональность фрагмента. Если аспект не преодолел порог, используется «общее впечатление». Это эвристика и она может ошибаться на отрицаниях, сложных связях и нескольких аспектах.

Ключ Kafka-сообщения – review_id, данные – UTF-8 JSON. reviews.created содержит review_id, clinic_id, rating, text, created_at и labels; reviews.indexed содержит review_id и имя индекса. Worker подтверждает смещение после индексации и отправки события. Полные примеры находятся в contracts/reviews.created.json и contracts/reviews.indexed.json. API описано в contracts/API.md и /docs.

Места применения паттернов: Pipeline – ReviewPipeline.run; Strategy – review_strategy и три реализации; Repository – PostgresReviewRepository.save; Adapter – HuggingFaceReviewClassifier; Observer – LocalReviewObserver.update. Observer локальной записи вызывается при обработке отзыва; Kafka worker является отдельным потребителем событий.

8 Данные, метрики и ограничения

Корпус helling100/sentiment-pragmatic-med-ru, ревизия 4e0223b289a187e9538c7056609f1cb5061119bb, лицензия CC BY-SA 4.0. По сохранённому файлу clinical_corpus_metrics.json после удаления дубликатов использованы 14 151 предложение для обучения, 3 058 для выбора порогов и 3 159 для теста. Исходные данные для повторного обучения загружаются скриптом train_clinical_corpus.py; модели входят в artifacts и требуют scikit-learn 1.9.1.

По пяти аспектам F1: врач 0,6157; ожидание 0,5051; сервис 0,5744; стоимость 0,5138; результат лечения 0,4660. Метки тональности заданы на уровне предложения, а не каждого аспекта. Повторная оценка всего корпуса в текущем запуске не выполнялась; приведены сохранённые результаты и методика из проекта.

Во время Docker-проверки модель ошибочно определила ожидание как positive во фразе «Врач внимательный, но долгое ожидание было ужасным». Поэтому сводка отображает предсказания модели, а не проверенные мнения пациентов. Автоматического обезличивания нет; персональные данные необходимо удалить до загрузки.

Дополнительно в контейнере запущен Hugging Face адаптер blanchefort/rubert-base-cased-sentiment: на 15 учебных контрольных фразах accuracy и macro-F1 составили 0,6667. Режим transformer на фразе «Врач внимательный, но долго ждал приёма» вернул врач/positive и ожидание/negative. Показатели относятся к малой ручной выборке и не являются независимой оценкой качества. Подробные результаты сохранены в artifacts/transformer_metrics.json.

9 Результаты запуска и тестирования

В Docker Compose 29.10.2026 запущены API, worker, PostgreSQL, Elasticsearch и Kafka. Три демонстрационных отзыва отправлены потоком JSONL; POST вернул 201, GET и сводка – 200, пустой запрос – 422. Записи проверены в PostgreSQL и Elasticsearch; из Kafka получены события reviews.created и reviews.indexed. После обновления demo-001 в PostgreSQL сохранён новый текст и рейтинг; сводка не дублировала отзыв. Текстовые протоколы проверки находятся в artifacts/docker_*.txt и docker_api_summary.json. Повторно проверен запуск через Запуск_Docker.ps1 из папки с кириллическим именем: после down и повторного up запись demo-001 и сводка сохранились в PostgreSQL; данные и индекс Elasticsearch расположены в именованных томах. Kafka в текущей конфигурации не сохраняет события после пересоздания брокера; после повторного старта публикация и чтение обоих топиков подтверждены заново.

Для проверки использован host-порт 18013, внутренний порт API 8013. Локальные и контейнерные тесты: 8/8 успешно. Контейнер использовал scikit-learn 1.9.1, соответствующий версии сохранённых моделей. На локальной системе с версией 1.8.0 Python выдавал предупреждение о несовпадении версий, хотя тесты прошли.

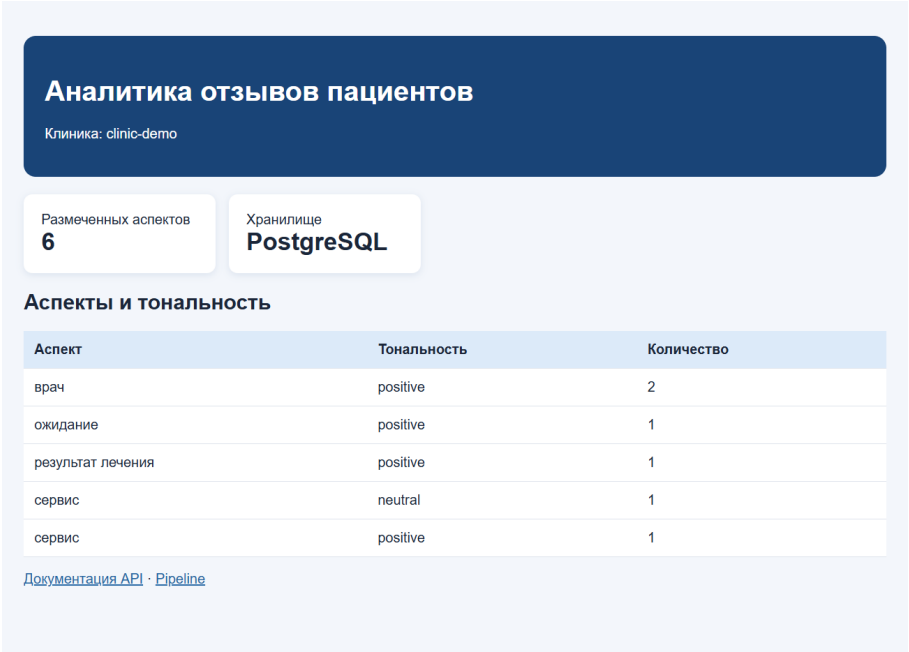


Рисунок 8 - Dashboard контейнерного приложения после обработки отзывов

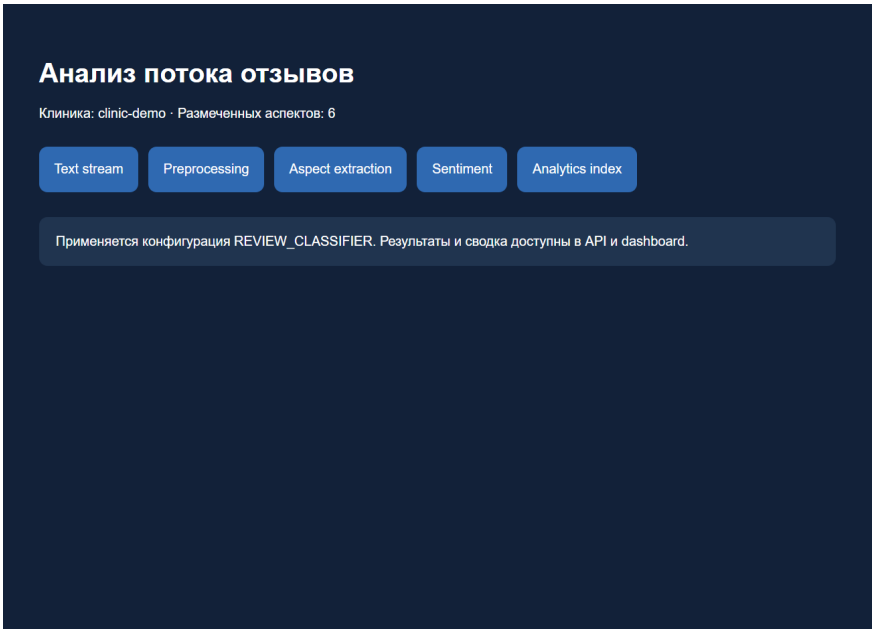


Рисунок 9 - Экран этапов обработки потока отзывов

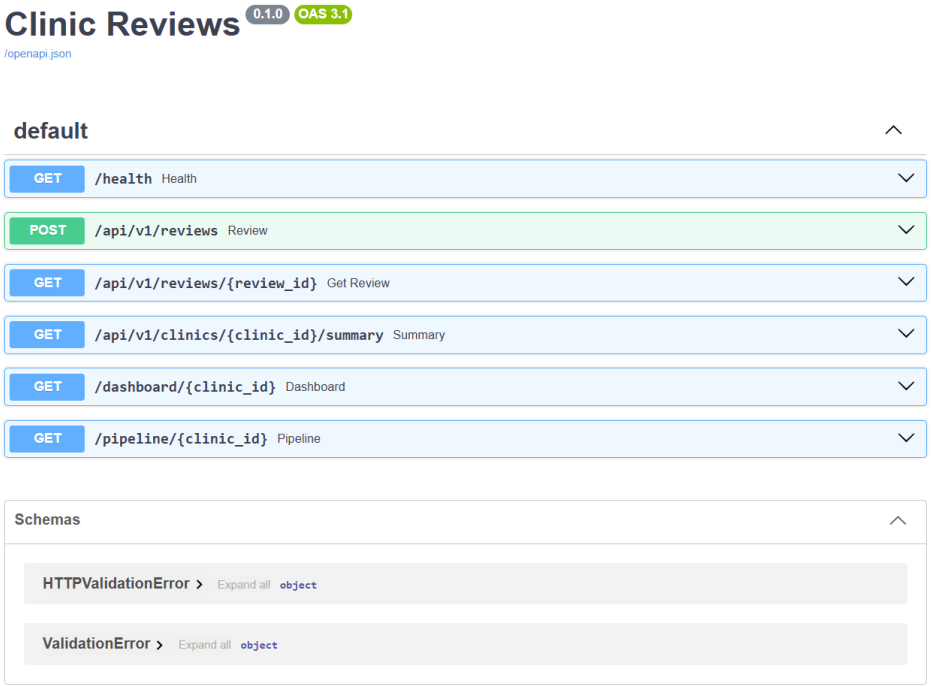


Рисунок 10 - Документация API запущенного приложения

Распределение работы: постановка, проектирование, разработка, проверка и подготовка отчёта выполняются исполнителем (мной)

Заключение

Разработан сервис анализа обезличенных отзывов с выделением аспектов, определением тональности, сохранением разметки и публикацией событий.

Список литературных источников

1. Apache Kafka Documentation [Электронный ресурс]. URL: https://kafka.apache.org/documentation/ (дата обращения: 26.10.2026).

2. Docker Docs [Электронный ресурс]. URL: https://docs.docker.com/ (дата обращения: 26.10.2026).

3. FastAPI Documentation [Электронный ресурс]. URL: https://fastapi.tiangolo.com/ (дата обращения: 26.10.2026).

4. PyTorch Documentation [Электронный ресурс]. URL: https://pytorch.org/docs/ (дата обращения: 26.10.2026).

ПРИЛОЖЕНИЕ А. Результаты проверки

Таблица А.1 - Результаты и ограничения проверки

Показатель	**Содержание**
Локально подтверждено	8 модульных тестов; сохранённые результаты корпуса: macro-F1 аспектов 0,5350, тональности 0,7333.
Production-контур в исходниках	Hugging Face, Kafka, PostgreSQL, Elasticsearch adapters; services.yml
Граница проверки	Оценка тональности относится к предложениям; см. приложение Б.

ПРИЛОЖЕНИЕ Б. Дополнительные результаты

Отзывы. Корпус helling100/sentiment-pragmatic-med-ru: 14 151 предложение для обучения, 3 058 для подбора порогов и 3 159 для теста. После удаления совпадений между частями macro-F1 по пяти аспектам – 0,5350 против 0,3397 у словарного baseline; по тональности – 0,7333 против 0,3325. Метки тональности относятся к предложению; отдельной проверки тональности каждого аспекта нет. На фразе «Лечение не помогло» модель ошибочно предсказала положительную тональность.

Источник данных: https://huggingface.co/datasets/helling100/sentiment-pragmatic-med-ru

ПРИЛОЖЕНИЕ В. Развёртывание

Dockerfile: Отзывы пациентов

```
FROM python:3.12-slim
WORKDIR /app
ENV PIP_DEFAULT_TIMEOUT=180 PIP_RETRIES=5
RUN pip install --no-cache-dir --extra-index-url
https://download.pytorch.org/whl/cpu torch==2.14.1+cpu
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8013
CMD ["uvicorn", "api:api", "--host", "0.0.0.0", "--port",
"8013"]
```

Порядок запуска в PowerShell

Откройте PowerShell в папке проекта. Для запуска из пути с русскими буквами предусмотрен скрипт Запуск_Docker.ps1: он создаёт временную копию с ASCII-путём, запускает compose на порту 18013 и выводит команду остановки. Альтернативно используйте services.yml в каталоге с ASCII-путём. Для запуска выполните powershell -NoProfile -ExecutionPolicy Bypass -File .\Запуск_Docker.ps1. Скрипт выводит адрес API и точную команду остановки; тома сохраняются при down.

```
docker compose -f services.yml config
docker compose -f services.yml up --build -d --wait
docker compose -f services.yml ps
python -X utf8 -m unittest -v
docker compose -f services.yml down
```